Design Model::Main

Trabalho Jogo++

Listas

«Encadeamento Simples»
Lista
-pPrimeiro: *Elemento<TL>
-pUltimo: *Elemento<TL>
+create+-Lista()
+destroy+-Lista()
+incluirElemento(p: *TL): void
+removerElemento(): void
+limpar(): void
+getPrimeiro(): Elemento<TL>*

TL: Class

0..1
+Conhecer Próximo

TE: Class

Elemento
-pProx: *Elemento<TE>
-pinfo: *TE
+create+-Elemento()
+destroy+-Elemento()
+incluir(p: *TE): void
+setProximo(pE: *Elemento<TE>)
+const getProximo(): *Elemento<TE>
+getInfo(): TE*

0..1
+Conhecer Primeiro
0..1
+Conhecer Ultimo
0..1

TE: Class

Iterator
+atual: *Elemento<TE>
«create»+Iterator(c: Element<TE>* = NULL)
«destroy»+Iterator()
+operator++(): Iterator&
+operator++(in: int): Iterator&
+operator==(outro: const Iterator&): bool const
+operator!=(outro: const Iterator&): bool const
+operator=(outro: const Iterator&): void
+operator*(): TE* const
+const get_atual(): Elemento<TE>*

depende

«Coleção»
ListaEntidades
-LEs: Lista<Entidade>
«create»+ListaEntidades()
«destroy»+ListaEntidades()
+incluir(pE: *Entidade): void
+remover(e: *Entidade): void
+percorreLista(): void
+desenharLista(): void
+getPrimeiro(): Lista<Entidade>*

«Abstrata»
Ente
#id: int
#pGG: static *Gerenciador_Grafico
#body: RectangleShape
#texturaEntidade: *Figura
«create»+Ente(pos: Vector2f, tam: Vector2f, textura: const string&)
«destroy»+virtual ~Ente()
+setTamanho(tam: Vector2f): void
+setPosicao(pos: Vector2f): void
+const getTamanho(): const Vector2f
+const getPosicao(): const Vector2f
+const getTempo(): float
+setTextura(textura: const string&): void
+const getBody(): const RectangleShape&
+const getBounds(): const FloatRect
+executar(): *virtual void = 0*
+desenhar(): void
+desenhar(t: Text): void
+static setGG(Gerenciador_Grafico: *pG)

+utilizar

Gerenciadores

«infraestrutura - SFML»
Gerenciador_Grafico
-window: RenderWindow*
-relogio: Clock
-tempo: float
-pGrafico: static GerenciadorGrafico*
«create»-Gerenciador_Grafico()
«destroy»+Gerenciador_Grafico()
+getGerenciadorGrafico(): static GerenciadorGrafico*
+getWindow(): RenderWindow*
+getWindowTam(): Vector2f
+getWindowCentro(): Vector2f
+getTempo(): float
+resetarRelogio(): void
+fecharJanela(): void
+verificaJanelaAberta(): const bool
+desenharEnte(pE: Ente*): void
+desenharTexto(pT: Text): void

+instancia

«Mediador - STL»
Gerenciador_Colisões
-jogador1: *Jogador
-jogador2: *Jogador
-LIs: vector<*Inimigo>
-LOs: list<*Obstaculo>
-LPs: set<*Projetil>
«create»+GerenciadorColisoes()
«destroy»+GerenciadorColisoes()
-const verificarColisao(pe1: *Entidade, pe2: *Entidade): const bool
+setJogadores(pJ: Jogador*): void
+incluirInimigo(pi: *Inimigo): void
+incluirObstaculo(po: *Obstaculo): void
+incluirProjetil(pj: *Projetil): void
+tratarColisoesJogsInimigs(): void
+tratarColisoesJogsObstacs(): void
+tratarColisoesInimigsObstacs(): void
+tratarColisaoChao(pEnt: Entidade*): void
+tratarColisaoEntsChao(): void
+tratarColisaoObstacProjetil(): void
+tratarColisaoInimigsProjetil(): void
+executar(): void
-tratarColisoesJogsProjeteis(): void

1

«Singleton»
Gerenciador_Eventos
-pGrafico: static GerenciadorGrafico*
-pJog: Jogador*
-pJog2: Jogador*
-menu: Menu*
-pEventos: static GerenciadorEventos*
«Create»-GerenciadorEventos()
«destroy»+GerenciadorEventos()
+getGerenciadorEventos(): static GerenciadorEventos*
+setJogador(pJ: Jogador*): void
+deletarJogadores(): void
+setMenu(m: Menu*): void
+verificaTeclaPressionada(tecla: Key): void
+verificaTeclaSolta(tecla: Key): void
+executar(): void

+instancia

+gerencia

2

+gerenciar

Entidades

«Abstrata»
Entidade
#velocidade: Vector2f
#noChao: bool
#empuxo: float
#buffer: ostream
«create»+Entidade(pos: Vector2f, tam: Vector2f, textura: const string&, v: Vector2f, e: float)
«destroy»+virtual ~Entidade()
+setVelocidade(v: Vector2f): void
+const getVelocidade(): const Vector2f
+gravitar(): void
+setnoChao(nc: bool): void
+const getnoChao(): const bool
+movimentar(m: Vector2f): void
+executar(): virtual void = 0
#salvarDataBuffer(): void
+salvar(): virtual void = 0

+gerenciar

«Entidade»
Projetil
#personagem
#pPersonagem: Personagem*
#alvo: Entidade*
#offset: Vector2f
#cravado: bool
#tempoCravafo: float
«create»+Projetil(pos: Vector2f, tam: Vector2f, textura: const string&, v: Vector2f, e: float)
«destroy»+Projetil()
+setPersonagem(pP: Personagem*): void
+getPersonagem(): Personagem*
+reiniciarProjetil(): void
+tratarAlvoNocauteado(): void
+setAtivo(a: bool): void
+getAtivo(): bool
+verificaForadaTela(): void
+cravarProjetil(e: Entidade*): void
+mover(): void
+executar(): void
+salvar(): void

+Arremessar

realiza

Fases

«Abstratata - Ente»
Fase
#ListaEntFase: ListaEntidades
#lJogs: ListaEntidades
#GC: GerenciadorColisoes
#pJogador: Jogador*
#pJogador2: Jogador*
«create»+Fase(pJ: Jogador*, pJ2: Jogador*, pos: Vector2f, tam: Vector2f, textura: const string&)
«destroy»+Fase()
#criarInimigos(): virtual void = 0
#criarChao(): void
#criarObstaculo(): virtual void = 0
#criarPlataformas()
#criarProjeteis(): virtual void = 0
#criarCenario(): void
+getListaEntFase(): ListaEntidades*
+executar(): virtual void = 0
+atualizaLogs(): void
+desenhar(): void

«Fase 1»
Fase Primeira
-maxinimMedios: const int
«Create»+Fase_Primeira(pJ: Jogador*, pJ2: Jogador*, pos: Vector2f, tam: Vector2f, textura: const string&)
«Destroy»+Fase_Primeira()
+criarInimigos(): void
+criarDragoes(): void
+criarObstaculos(): void
+criarBlocosMagma(): void
+criarProjeteis(): void
+executar(): void

+conhecer

«Fase 2»
Fase Segunda
-maxChefoes: const int
«Create»+Fase_Segunda(pJ: Jogador*, pJ2: Jogador*, pos: Vector2f, tam: Vector2f, textura: const string&)
«Destroy»+Fase_Segunda()
+criarInimigos(): void
+criarNinjas(): void
+criarObstaculos(): void
+criarEspinhos(): void
+criarProjeteis(): void

realiza

Obstaculos

«Entidade - Abstrata»
Obstaculo
#danoso: bool
«create»+Obstaculo(pos: Vector2f, tam: Vector2f, textura: const string&, v: Vector2f, d: bool, e: float)
«destroy»+Obstaculo()
+executar(): virtual void = 0
+obstaculizar(p: *Jogador): *virtual void = 0*
+salvarDataBuffer(): void
+salvar(): virtual void = 0

«Obstaculo1»
Plataforma
-coeficienteAtrito: float: int
«create»+Plataforma(pos: Vector2f, tam: Vector2f, textura: const string&, v: Vector2f, d: bool, e: float)
«destroy»+Plataforma()
+executar(): void
+obstaculizar(p: Personagem*): void

«Obstaculo 2»
Magma_Block
-yOriginal: float
-yLimite: float
+velocidadeMovimento: float
+pgadorEmCima: bool
+cooldownDano: float
+tempoSemJogador: float
«create»+Magma_Block(pos: Vector2f, tam: Vector2f, textura: const string&, v: Vector2f, d: bool, e: float)
«destroy»+Magma_Block()
+obstaculizar(p: Personagem*): void
+executar()

«Obstaculo Difícil»
Espinho
-danosidade: short int
«Create»+Espinho(pos: Vector2f, tam: Vector2f, textura: const string&, v: Vector2f, d: bool, dano: short int, e: float)
«Destroy»+Espinho()
+const getDanosidade(): const short int
+executar()
+obstaculizar(p: Personagem*): void
+salvarDataBuffer(): void

«PRINCIPAL»
Jogo
-pJogador: Jogador
-pJogador2: Jogador
-pontuacaoFinalP1: int
-pontuacaoFinalP2: int
-ranking: vector<Ranking>
-numJogadores: int
-faseAtual: int
-GG: GerenciadorGrafico*
-GE: GerenciadorEventos*
-fase1: Fase_Primeira
-fase2: Fase_Segunda
-menu: Menu*
«create»+Jogo()
«destroy»+Jogo()
+setNumJogadores(n: int): void
+getNumJogadores(): int
+getFaseAtual(): int
+salvarPontuacao(nome: const string&, jogador: int)
+carregarRanking(): void
+const getRanking(): const vector<Ranking>&
+getPontuacaoFinal(j: int): int
+getVidaJogador(j: int): int
+getPontuacaoJogador(j: int): int
+criarFasePrimeira(): void
+deletarFasePrimeira(): void
+criarFaseSegunda(): void
+deletarFaseSegunda(): void
+executarMenu(): void
+executar(): void

Personagens

«Entidade - Abstrata»
Personagem
#numvidas: int
#velocidadeKnockback: Vector2f
#cooldownKnockback: float
#esquerda: bool
#direita: bool
#tempoNoChao: float
#tempoInvulneravel: float
«create»+Personagem(pos: Vector2f, tam: Vector2f, textura: const string&, v: Vector2f, e: float, n: int)
«destroy»+Personagem()
+const getVida(): const int
+const getVivo(): const bool
+aplicarKnockback(forca: Vector2f): void
+recebeDano(dano: int): void
+mover(): virtual void = 0
+executar(): virtual void = 0
+const getTempoInvulneravel(): const float
+setTempoInvulneravel(t: float): void
+setDirecao(d: Direcao): void
+GetDirecao(): Direcao
+salvarDataBuffer(): void
+salvar(): virtual void = 0

«Personagem»
Jogador
-movDir: bool
-movEsq: bool
-querAtirar: bool
-coolDownTiro: float
+pontuacao: int
«create»+Jogador(pos: Vector2f, tam: Vector2f, textura: const string&, v: Vector2f, e: float, n: int)
«destroy»+Jogador()
+setMovDir(b: bool): void
+setMovEsq(b: bool): void
+atirar(): void
+setProjetil(p: Projetil*): void
+getProjetilAtivo(): bool
+setCooldownTiro(cd: float): void
+getCooldownTiro(): float
+mover(): void
+pular(): void
+colidir(pIn: Inimigo*)
+executar(): void
+salvar(): void

1..2

«Personagem Abstrata»
Inimigo
#cooldownMovimento: float
#movimento: int
#lJogs: ListaEntidades*
#nivel_maldade: int
«Create»+Inimigo(pos: Vector2f, tam: Vector2f, textura: const string&, v: Vector2f, e: float, n: int, nm: int)
«Destroy»+Inimigo()
+danificar(p: *Jogador): *virtual void = 0*
+set CooldownMovimento(c: float): void
+getCooldownMovimento(): float
+recebeDano(dano: int): vpod
+getNivelMaldade(): int
+perseguir(pJog: Jogador*): virtual void = 0
+movimentoAleatorio(): virtual void = 0
+setMovimento(m: int): void
+getMovimento(): int
+setListaJogadores(lJ: ListaEntidades*): void
+executar(): virtual void = 0
+mover(): virtual void = 0
+salve(): virtual void = 0
+salvarDataBuffer(): void

«Inimigo 2 - Médio»
Dragão
-altura_voo: float
-altura_default: float
-diving: bool
-delay_troca_direcao: float
-direcao: int
«Create»+Inimigo_Medio(pos: Vector2f, tam: Vector2f, textura: const string&, v: Vector2f, e: float, a: float, n: int, nm: int)
«Destroy»+Inimigo_Medio()
+danificar(p: *Jogador): void
+mover(): void
+executar(): void
+perseguir(pJog: Jogador*): void
+movimentoAleatorio(): void
+salvar()

3..*

«Inimigo 3 - Chefão»
Ninja
-querAtirar: bool
-coolDownTiro: float
«Create»+Ninja<<Create>>Inimigo_Medio(in pos:Vector2f, in tam:Vector2f, in textura:const string&, in v:Vector2f, in e:float, in a:float, in n:int, in nm:int)(): void
«Destroy»+Ninja()
+setQuerAtirar(a: bool): void
+getQuerAtirar(): bool
+setCooldownTiro(cd: float): void
+getCooldownTiro(): float
+atirar(): void
+mover(): void
+movimentoAleatorio(): void
+perseguir(pJog: Jogador*): void
+danificar(p: *Jogador): void
+executar(): void

3..*

«Inimigo 1 - Fácil»
Oni
-raio: float
«Create»+Inimigo_Facil(pos: Vector2f, tam: Vector2f, textura: const string&, v: Vector2f, e: float, n: int, nm: int)
«Destroy»+Inimigo_Facil()
+danificar(p: *Jogador): void
+mover(): void
+movimentoAleatorio(): void
+perseguir(pJog: Jogador*): void
+executar(): void

3..*

«Ente»
Menu
-pJogo: Jogo*
-menuAtual: tipoMenu
-vecBotao: vector<Botao*>
-botaoSelecionado: iterator
-vecTexto: vector<Text>
-idxVidaP1: int
-idxVidaP2: int
-idxPontuacaoP1: int
-idxPontuacaoP2: int
-nomeDigitado: string
+idxNomeJogador1: int
+jogadorSalvando: int
-fonte: Font
«create»+Menu(pos: Vector2f, tam: Vector2f, textura: const string&)
«destroy»+Menu()
+setJogo(pJ: Jogo*): void
+mudarMenu(menu: tipoMenu): void
+getTipoMenu(): tipoMenu
+digitarNome(unicode: Uint32): void
+centralizarTexto(Texto: Text&): void
+limparvecBotao(): void
+limparvecTexto(): void
+limparMenu(): void
+void adicionarTexto(s: String, pos: Vector2f, tam: unsigned int): void
+criarHUDJogo(): void
+criarMenuPrincipal(): void
+criarMenuRanking(): void
+criarMenuFases(): void
+criarMenuSelecionarJogadores(): coid
+criarMenuPausa(): void
+criarMenuGameOver(): void
+criarMenuSelecionarJogadores(): void
+salvarPontuacao(): void
+selecionarBotoes(d: Direcao): void
+executarBotao(): void
+executar(): void

«enumeration»
tipoMenu
NO_JOGO
MENU_PRINCIPAL
MENU_PAUSA
MENU_FASES
MENU_OPCOES
MENU_GAME_OVER
MENU_SALVAR_PONTUACAO
MENU_RANKING

+1..*

«enumeration»
AcaoBotao
ACAO_JOGAR_FASE_1
ACAO_JOGAR_FASE_2
ACAO_1_JOGADOR
ACAO_2_JOGADORES
ACAO_MENU_JOGADORES
ACAO_SAIR
ACAO_MENU_FASES
ACAO_VOLTAR
ACAO_VOLTAR_MENU_JOGADORES
ACAO_VOLTAR_MENU_GAME_OVER
ACAO_RESUMIR
ACAO_MENU_SALVAR_PONTUACAO
ACAO_SALVAR_PONTUACAO
ACAO_MENU_RANKING

Botão
-texto: sf::Text
-fonte: sf::Font
-acao: AcaoBotao
-selecionado: bool
-tempoAnimacao: float
«Create»+Botao(a: AcaoBotao, str: const String&, pos: Vector2f, tam: Vector2f, textura: const string&)
«Destroy»+Botao()
+const getAcao(): AcaoBotao
+centralizarTexto(): void
+setSelecionado(s: bool): void
+getSelecionado(): bool
+executar(): void

Naturalmente, as classes folhas nas hierarquias teriam seus nomes refinados, inclusive... Neste sentido, os atributos indicados também podem ser alterados, modificado etc nelas.

Eis um diagrama em fase inicial que começa a atender os requisitos postos e deve ser extendido e finalizado para tal.

A partir desta classes, seus relacionamentos e afins, criar outras visando atender os requisitos postos.